Installing meetingrec on a second machine

Everything needed to get this running on a fresh Windows box, in order.
Budget about 20 minutes, most of it waiting on downloads.

Part 1 — On the machine that already works

You need to get the project across. Git bundles are the cleanest way: one
file, full history, no GitHub account or SSH keys.

```
cd "$HOME\dev\meetingrec"

# make sure nothing is uncommitted
git add -A
git commit -m "pre-transfer snapshot"

# one file containing the entire repo and its history
git bundle create "G:\My Drive\meetingrec.bundle" --all
```

Wait for Drive to finish syncing — the tray icon stops showing activity.

Do not copy `config.toml`. It holds device names that will not exist on the
other machine, and a wrong `mic_device` is the single most common way to end up
recording silence. Part 4 builds a fresh one.

Part 2 — Prerequisites on the new machine

Python

```
py --version
```

If that fails, install from **python.org** (not the Microsoft Store — the Store
build sandboxes registry and filesystem access, and this app reads `HKCU`
directly). Tick **Add python.exe to PATH** on the first installer screen.

Needs 3.11 or newer for `tomllib`.

Git

```
git --version
```

If missing:

```
winget install --id Git.Git -e --source winget
```

Close and reopen PowerShell afterwards so PATH refreshes.

Google Drive for Desktop

Required for the default storage backend. Install from google.com/drive/download,
sign in, and confirm you get a drive letter:

```
Test-Path "G:\My Drive"
```

If Drive mounts somewhere other than `G:`, that is fine — the app detects it.

Part 3 — Install the project

```
mkdir "$HOME\dev" -Force | Out-Null
cd "$HOME\dev"
git clone "G:\My Drive\meetingrec.bundle"
meetingrec
cd meetingrec
git remote remove origin          # the bundle is
a snapshot, not a live remote
powershell -ExecutionPolicy Bypass -File
.\bootstrap.ps1
```

`bootstrap.ps1` creates the venv, installs the package and its dependencies,
copies `config.example.toml` to `config.toml`, and runs the test suite.

Expect **70 passed**. Anything less means a dependency failed to install — the
pip output above it will say which.

If a wheel fails to build

You are on x86-64 here, so this is unlikely — the ARM machine is the fussy one.
If `soundcard` or `sounddevice` fails anyway:

```
.\.venv\Scripts\python.exe -m pip install --
upgrade pip setuptools wheel
.\.venv\Scripts\python.exe -m pip install -e ".
[detect,dev]" --no-cache-dir
```

Part 4 — Configure for this machine's hardware

This part matters more than the rest. Two of the hardest bugs in this project
were wrong device selection, not code.

Find the devices

```
.\.venv\Scripts\python.exe -m meetingrec devices
```

Note the exact name of the mic you will actually wear, and confirm it shows
`[works]`. A device listed by Windows is not necessarily one that opens.

Write the config

Open `config.toml` and set, under `[capture]`:

```
# In priority order. Put the device you actually
wear first — if the first
# one is a dongle whose headset is switched off,
```

```
it opens fine and records
# pure silence. There is a watchdog for that now,
but it costs 12 seconds.
mic_device = ["<exact name from devices output>",
"<backup device>"]

# 1024 frames (~21ms). Measured: 4096 gave 9-26%
digital silence on a
# Bluetooth mic, 1024 gave 0.2%. Large buffers
starve between packets.
block_frames = 1024

# Let a lazily-opened mic settle before the file
opens. A fresh Bluetooth
# stream drops ~20% of its first seconds.
warmup_seconds = 1.5

# 300 suits a headset mic; 120-150 suits a wired
mic close to your mouth.
# Too high sounds thin, too low sounds boomy. Tune
after listening.
highpass_hz = 300.0
```

Leave everything else at its default.

Microphones go through MME, not WASAPI — that is in the code, not the
config, but it is the single change that fixed audio quality. Raw WASAPI
bypasses the Windows audio engine and with it the endpoint's own capture
processing; on a Bluetooth headset that was the difference between 34% and 77%
speech-band energy.

Part 5 — Verify, in this order

Each step proves one thing. Do not skip ahead; a failure later is much harder
to diagnose.

```
# 1. archive target reachable — writes a throwaway
file to Drive
.\.venv\Scripts\python.exe -m meetingrec storage

# 2. audio path — play music and talk during this
.\.venv\Scripts\python.exe -m meetingrec
testrecord --seconds 20

# 3. detection — run during a real call, look for
"<== MIC OPEN NOW"
.\.venv\Scripts\python.exe -m meetingrec list
```

After step 2, **listen to all three files** in `recordings\`. `-mic.wav` should
be your voice only, `-system.wav` everyone else, `-mix.wav` both. The summary
also prints dropout percentage and speech-band figures.

For step 3, if your meeting app is not in the default allowlist, add it to
`config.toml` under `[detection]`. The list replaces the defaults entirely, so
include everything you use:

```
allowlist = ["msteams_8wekyb3d8bbwe", "teams.exe",
"ms-teams.exe", "zoom.exe",
             "slack.exe", "chrome.exe",
"msedge.exe"]
```

Part 6 — Daily use

```
cd "$HOME\dev\meetingrec"
.\.venv\Scripts\python.exe -m meetingrec run
```

Leave the window open. Minimising is fine; closing kills it and loses anything
mid-recording. Meetings are detected, recorded, cleaned, aligned and archived
to `G:\My Drive\Meeting Recordings\<machine>\<year>\<month>\` without further
input.

Shortcut

```
notepad $PROFILE
```

Add:

```
function meetingrec {
    Push-Location "$HOME\dev\meetingrec"
    & .\.venv\Scripts\python.exe -m meetingrec
@args
    Pop-Location
}
```

Then `meetingrec run`, `meetingrec devices`, and so on work from anywhere.

Start automatically (optional)

Only once you trust it — `pythonw.exe` hides the console, and the console is

how you notice a dead device.

Task Scheduler, **At log on**, **Run only when user is logged on**. Not "run
whether logged on or not": that has no interactive token, and this app needs
`HKCU` and per-session audio endpoints, both of which are invisible from
session 0. A Windows service fails for the same reason.

Troubleshooting

Symptom	Cause	Fix
`python not recognized`	Store alias intercepting	Use `py`, or disable the alias in Settings → Apps → Advanced → App execution aliases
Tests fail on import	Dependency missing	Re-run `pip install -e ".[detect,dev]"` and read the error
`devices` shows no loopback	soundcard not installed	Part 3
Mic track is silent	Wrong or powered-off device	Check `mic_device` against `devices` output

Symptom	Cause	Fix
System track is silent	Output went to a different endpoint	Set the headset as Windows default for playback
Heavy dropouts	Bluetooth interference	Move the dongle off a USB 3 port; try a USB extension
Muffled audio	`highpass_hz` too low	Raise toward 300
Thin, telephone-ish audio	`highpass_hz` too high	Lower toward 150
Nothing archived	Under `min_session_seconds`	Short meetings are deleted by design; lower the setting
Tracks out of sync	Pre-roll offset, pre-alignment recording	`scripts\realign.py --probe`, then `--offset <n>`

What is not solved

Bluetooth packet loss is real and environmental — expect a few percent of any
recording to be reconstructed, and occasional bursts up to 20% when the 2.4 GHz
band is busy. Concealment hides the clicks; it cannot recover lost words.

HFP narrowband is a hard ceiling on any Bluetooth headset. Roughly 16 kHz of
bandwidth, telephone quality. A wired USB microphone is the only way past it.